

LAPORAN DATA WAREHOUSE DAN INTELLIGENCE BUSINESS

Pipeline Analytic Data NYC Taxi



Oleh:

I Putu Yosua Yerikho

24/552518/PPA/07012

**PROGRAM MAGISTER ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA**

2025

Table of Contents

A. Introduction.....	1
Business Context.....	1
Dataset Description.....	1
B. Tech Stack.....	1
Folder Structure.....	2
C. Implementation Detail.....	3
.env File.....	3
Running Dag.....	3
Processing.....	4
nyc_taxi_ingestion.....	4
nyc_taxi_transform.....	4
nyc_taxi_monitor.....	5
Email Notification.....	5
Validation and Testing.....	5
D. Data Visualization.....	7
Dashboard.....	7
E. Airflow Diagrams.....	8
Workflows.....	8
Raw Data Source.....	9
DAG: NYC Taxi Ingestion.....	9
DAG: NYC Taxi Transform.....	9
DAG: NYC Taxi Monitor.....	10
Dashboard and Analysis.....	10
System Notification.....	10
Gantt Chart.....	11
DAG: NYC Taxi Ingestion.....	11
DAG: NYC Taxi Transform.....	11
DAG: NYC Taxi Monitor.....	11
Task Flow.....	12
DAG: NYC Taxi Ingestion.....	12
DAG: NYC Taxi Transform.....	12
DAG: NYC Taxi Monitor.....	12
Lineage Graph.....	13
F. Summary.....	13
Data Analysis.....	13
KPI Summary.....	13

Trend Analysis.....	14
Top Zones.....	14
Demand Heatmap.....	15
Payment Breakdown.....	15
Conclusion dan Suggestions.....	16
Conclusion.....	16
Suggestions.....	16

A. Introduction

Business Context

New York City memiliki lanskap dinamis untuk transportasi di perkotaan, dimana hal ini dapat menghasilkan data dalam jumlah besar yang memiliki nilai ekonomi dan operasional. Hal ini dapat diposisikan dalam konteks untuk memaksimalkan efisiensi, transparansi dan profitabilitas industri transportasi dalam bidang taksi. Dengan memproses data dari Data Trip Taksi NYC yang memiliki cukup banyak data perjalanan, termasuk waktu penjemputan, lokasi, tarif dan jarak. Proses data yang telah dilakukan bertujuan untuk mengubah log operasional menjadi aset data strategis. Tujuan utamanya adalah untuk mengoptimalkan penyampaian layanan dan alokasi sumber daya yang dapat dilakukan oleh para pemangku kepentingan.

Implementasi pipeline yang kuat dan terotomatisasi menggunakan **airflow** dan **dbt** dapat menjawab kebutuhan bisnis yang cukup krusial dimana dapat beralih dari analisis manual menjadi lebih terkelola dan terukur. Skalabilitas ini sangat penting karena volume data perjalanan sangat besar dan terus meningkat. Output dari analitik ini divisualisasikan ke dalam *dashboard* interaktif yang dapat digunakan untuk memantau data yang telah dikelola secara terukur. Kemampuan untuk memantau indikator kinerja utama yang mendekati waktu nyata memungkinkan untuk identifikasi cepat anomali baik itu melihat penurunan kualitas layanan atau memanfaatkan tren pasar.

Dataset Description

Dataset didapatkan melalui <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page> dimana pada analytic ini menggunakan dataset yang diambil secara 3 bulan berturut turut. Dataset yang telah diambil tersebut berbentuk .parquet yang ukuran filenya sekitar 40 - 50MB.

Selain dataset berbentuk .paquet, digunakan pula dataset zone lookup berbentuk .csv melalui link berikut https://d37ci6vzurychx.cloudfront.net/misc/taxi+_zone_lookup.csv. Dataset zone look up digunakan untuk memasukkan data zona.

B. Tech Stack

Beberapa tech stack yang digunakan pada project ini ditampilkan pada tabel dibawah ini :

Kategori	Teknologi	Versi	Deskripsi
Bahasa Pemrograman	Python	3.11	Digunakan untuk pemrograman dag

Transformasi dan Storage	Dbt Duck DB	1.9.6	Duck DB digunakan untuk data warehouse dan pemodelan data
Pemrosesan Data	Pandas	2.2.2	Digunakan untuk manipulasi data
Format Data	Pyarrow	22.0.0	Pemrosesan data berbentuk parquet
Visualisasi dan Dashboard	Dash	3.2.0	Membangun dashboard yang interaktif
	Dash Bootstrap Component	2.0.4	Membuat UI pada dashboard lebih menarik
Konfigurasi	Python Dotenv	1.2.1	Mengelola .env pada sistem

Folder Structure

```

.
├── Project Name/
│   ├── airflow/
│   │   ├── dags/
│   │   └── data/
│   │       ├── raw/
│   │       │   └── *.parquet
│   │       ├── databases/
│   │       ├── logs/
│   │       ├── queries/
│   │       ├── reports/
│   │       └── scripts/
│   └── dbt/
│       ├── analyses/
│       ├── logs/
│       ├── macros/
│       ├── models/
│       ├── seeds/
│       ├── snapshots/
│       ├── target/
│       └── tests/
├── documentations/
│   └── *.pdf
├── pages/
├── screenshots/
│   └── *.png

```

Struktur folder terdiri dari folder root utama yaitu nama project, dilanjutkan dengan airflow yang di dalam folder airflow berisikan data parquet dan script dags, kemudian dilanjutkan dengan folder dbt yang berisikan file - file dbt, documentations yang berisikan file pdf, pages yang berisikan dashboard UI dan terakhir folder screenshots yang berisikan hasil screenshots dari airflow UI.

C. Implementation Detail

.env File

```
# REMEMBER TO USE ABSOLUTE PATH HERE SUCH LIKE /Users/name/path/to/airflow
DATABASE_NAME="/Users/name/path/to/airflow/nyc_taxi.duckdb"
DATABASE_TABLE="yellow_tripdata"

SUMMARY_PATH="/Users/name/path/to/airflow/reports"

RAW_FOLDER="/Users/name/path/to/airflow/data/raw"

MAIL_MAILER=log
MAIL_HOST=127.0.0.1
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
MAIL_TO_ADDRESS="failure@example.com"
MAIL_FROM_ADDRESS="hello@example.com"
MAIL_FROM_NAME="from@example.com"

DAG_INGESTION_SCHEDULE="@monthly"
DAG_TRANSFORM_SCHEDULE="0 6 * * *"
DAG_MONITOR_SCHEDULE="0 */6 * * *"

DBT_ROOT_FOLDER="/Users/name/path/to/airflow/dbt"
```

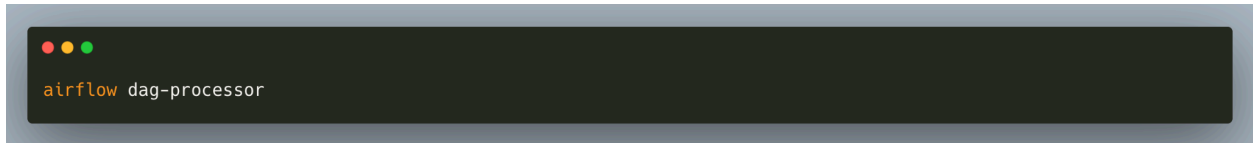
Copy lalu paste file `.env.example` menjadi `.env` kemudian ganti nilai variabel sesuai dengan yang diinginkan seperti email dan dag schedule, jangan lupa untuk memberikan absolute path kepada beberapa variabel sesuai dengan instruksi yang ada pada [README.md](#). Disarankan untuk `DATABASE_NAME=` diberikan path menuju folder dbt seperti `/Users/username/Sites/project/python/airflow_project/dbt/nyc_taxi.duckdb`.

Running Dag

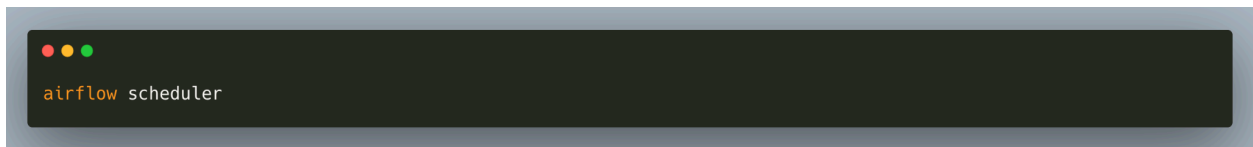
Setelah mengganti nilai variabel, selanjutnya mengganti menjalankan dags file dengan menggunakan tiga command berikut :

```
airflow api-server -p 8080
```

`airflow api-server` digunakan untuk menjalankan airflow dashboard UI yang bisa diakses melalui `http://localhost:8080`.



airflow dag-processor digunakan untuk menambah DAG yang telah kita buat kedalam airflow dashboard agar kita dapat menjalankan DAG tersebut.



airflow scheduler digunakan untuk menjalankan scheduler yang telah diaktifkan pada DAG di dashboard.

Processing

nyc_taxi_ingestion

Ketika masing - masing DAG dijalankan, DAG *nyc_taxi_ingestion* akan melakukan load data dari data yang telah diolah dari masing - masing file *.parquet* ke dalam *duck db* dan akan disimpan pada folder yang diinisialisasi pada file *.env*. Kemudian *nyc_taxi_ingestion* akan mengirimkan pesan error ataupun berhasil menuju email yang diberikan pada file *.env* ketika DAG selesai dieksekusi.

nyc_taxi_transform

DAG ini akan menjalankan *dbt* setiap jam enam pagi dimana *dbt* akan melakukan pengolahan data yang di dapat dari file *.duckdb* yang telah di load dari *nyc_taxi_ingestion* DAG. Setelah berhasil melakukan pengolahan data, DAG ini akan membuat lima tabel baru dan empat buah views baru pada *.duckdb* tadi. Masing - masing tabel dan views tersebut adalah sebagai berikut :

Tables

- *agg_daily_stats*
- *daily_anomaly_check*
- *dim_zones*
- *fct_trips*
- *taxi_zone_lookup*

Views

- *int_daily_metrics*
- *int_trips_enhanced*
- *stg_taxi_trips*

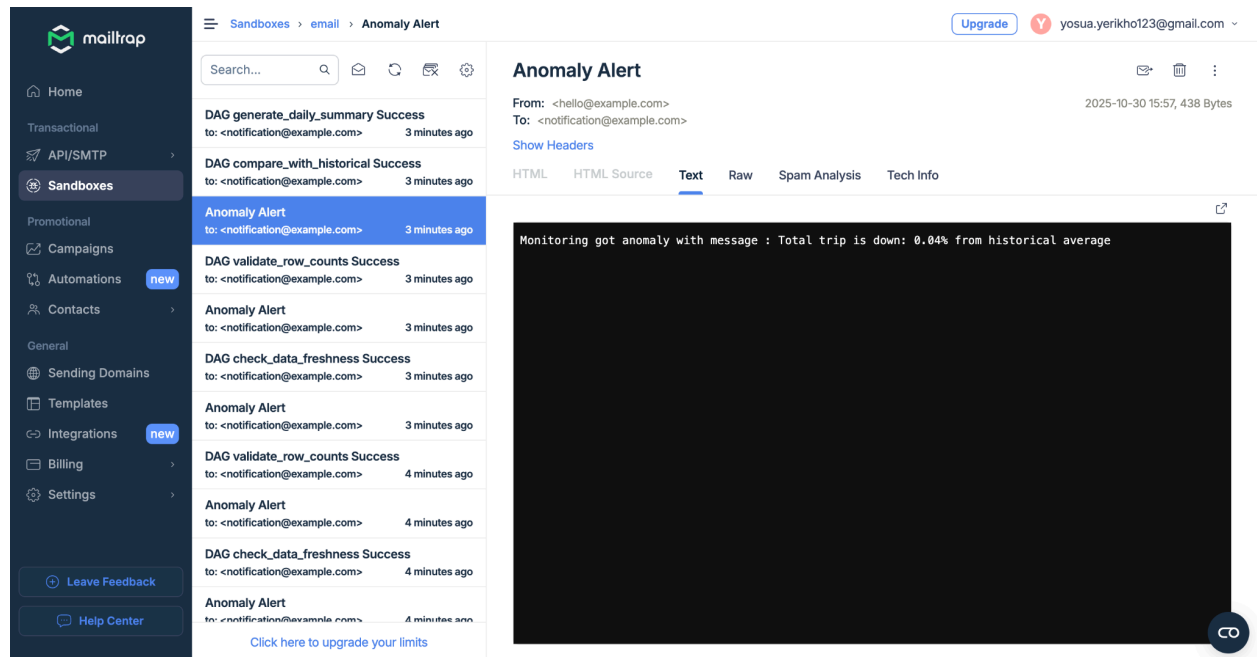
- stg_taxi_zones

nyc_taxi_monitor

DAG ini melakukan validasi anomali setiap enam jam untuk monitoring diantaranya adalah untuk *check_data_freshness*, *validate_row_counts*, *compare_with_historical* dan *generate_daily_summary*. Ketika ada anomali terdeteksi, maka sistem akan mengirimkan email kepada email tujuan yang telah diberikan pada file .env.

Email Notification

Masing - masing dari DAG tersebut ketika mengalami error ataupun success akan mengirimkan email notifikasi kepada email tujuan yang telah diberikan pada file .env. Contoh email notification dapat dilihat pada gambar dibawah ini :



Validation and Testing

Testing dan validasi dilakukan melalui DAG nyc_taxi_transform yang akan menjalankan command ``dbt test`` dan secara otomatis melakukan testing dengan menjalankan file .yaml yang ada pada folder ``./dbt/models/\*.yaml``. Masing - masing file .yaml tersebut dapat dilihat pada tabel berikut :

marts.yaml

Model	Deskripsi	Tipe Model	Data yang diuji
stg_taxi_zones	Staging table untuk	Staging	location_id (not null, unique),

	lookup zona taksi (hasil pembersihan dari seed CSV).		zone_name (not null)
stg_taxi_trips	Data mentah hasil pembersihan untuk setiap perjalanan taksi.	Staging	pickup_datetime (not_null), dropoff_datetime (not_null), pickup_location_id (not_null), dropoff_location_id (not_null)
int_trips_enhanced	Data perjalanan yang sudah diperkaya dengan zona dan metrik turunan.	Intermediate	avg_speed_mph (not_null)
fct_trips	Tabel fakta utama berisi satu baris per perjalanan.	Fact	dropoff_location_id (relasi ke dim_zones.location_id)
dim_zones	Tabel dimensi berisi metadata zona taksi.	Dimension	location_id (unique)
agg_daily_stats	Tabel agregasi harian untuk analisis tren dan KPI.	Aggregate	trip_date (not_null), total_trips (not_null), avg_fare (not_null)

monitoring.yml

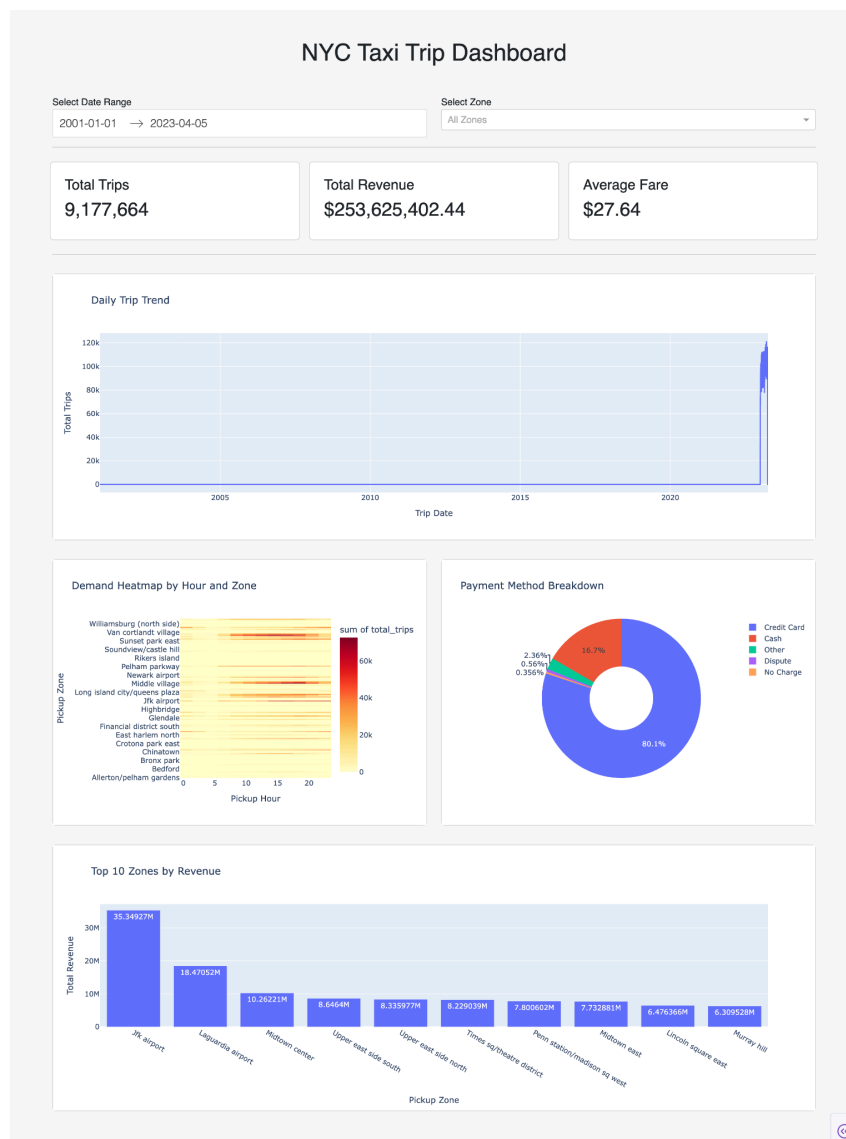
Model	Deskripsi	Tipe Model	Data yang diuji
daily_anomaly_check	Model yang berisi baris data yang dianggap anomali atau <i>outlier</i> pada data perjalanan harian.	Monitoring	Menggunakan singular test <code>dbt_utils.expression_is_true</code> dengan ekspresi <code>'COUNT(*) = 0'</code> . Jika model ini menghasilkan data, tes dbt akan gagal (ERROR).

D. Data Visualization

Dashboard

Visualisasi data dilakukan dengan membuat satu buah halaman dashboard dengan menggunakan library python yang bernama dash. Dash adalah sebuah framework open source yang dibuat oleh plotly yang digunakan untuk membangun aplikasi analitik interaktif dan dapat disesuaikan sesuai keinginan. Dash menjembatani analisis data kepada pengguna. Dashboard ini memiliki lima buah tampilan interaktif diantaranya sebagai berikut : KPI Summary, Trend Analysis, Top Zones, Demand Heatmap dan Payment Breakdown

Tampilan untuk masing - masing visualisasi tersebut dapat dilihat pada screenshots dibawah :



Dapat dilihat pada dashboard diatas, pengguna dapat melakukan filtering berdasarkan date range dan select zone, lalu dibawah filter tersebut terdapat tiga buah card yang mewakili KPI Summary seperti total trips, total revenue dan average fare. Lalu dibawah KPI Summary dapat dilihat Trend Analysis yang berupa grafik like. Selanjutnya ada dua buah grafik heatmap yang menggambarkan trip hour dan zona lalu payment method breakdown. Terakhir dashboard menampilkan top 10 zone by revenue.

E. Airflow Diagrams

Workflows

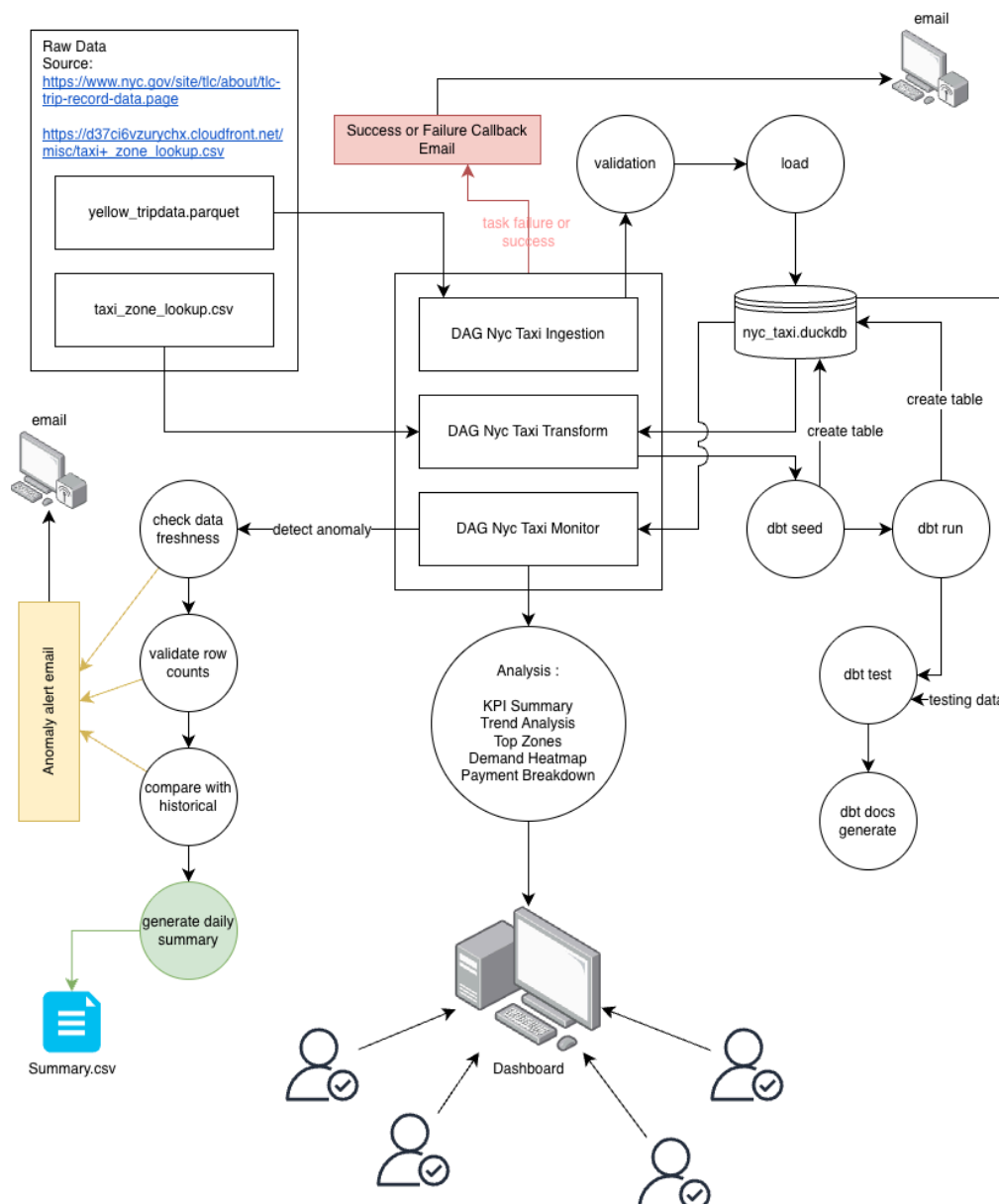


Diagram diatas menjelaskan mengenai alur Extract, Transform, Load dan monitoring otomatis untuk data New York City Taxi. Sistem ini dirancang agar setiap tahap pemrosesan data berjalan terjadwal dengan Airflow DAG, serta memiliki mekanisme notifikasi otomatis saat task berhasil atau gagal.

DAG Nyc Taxi Ingestion akan dijalankan setiap bulan, DAG Nyc Taxi Transform akan dijalankan setiap jam enam pagi dan DAG Nyc Taxi Monitor dijalankan tiap enam jam.

Raw Data Source

Bagian paling kiri pada diagram menunjukkan sumber data mentah yang diperoleh dari sumber berikut :

- Yellow Trip Data Parquet berisikan dataset utama yaitu transaksi taksi (pickup/dropoff, tarif, waktu, dan sebagainya).
- Taxi Zone Lookup CSV memuat tabel referensi yang berisi informasi zona pick up/drop off.

DAG: NYC Taxi Ingestion

DAG pertama ([nyc_taxi_ingestion](#)) bertugas melakukan pemuatan data mentah ke dalam database Duck DB.

Tahapan utamanya adalah sebagai berikut:

- [validation](#) akan memeriksa apakah file .parquet sudah valid (struktur kolom benar, tidak korup). Bila terjadi kegagalan, task akan memicu [on failure callback](#) yang kemudian akan mengirim email notifikasi kegagalan.
- [load](#) akan dijalankan jika validasi lolos, file dimuat ke tabel dalam [nyc_taxi.duckdb](#). Setelah berhasil, [onsuccess callback](#) mengirim email notifikasi bahwa tahap ingestion berhasil.

Pada tahapan ini, sistem akan memastikan bahwa data mentah telah berhasil masuk ke dalam database dengan baik.

DAG: NYC Taxi Transform

Tahap kedua adalah proses transformasi data menggunakan DBT (Data Build Tool). Tahapan utamanya meliputi:

- [dbt seed](#) akan mengimport data referensi seperti [taxi_zone_lookup.csv](#).
- [dbt run](#) akan menjalankan model transformasi (misalnya membersihkan kolom, menghitung metrik dan juga join antar tabel).
- [dbt test](#) akan melakukan uji integritas dan kualitas data.
- [dbt docs generate](#) akan menghasilkan dokumentasi model DBT.

Semua hasil data yang telah ditransformasi akan disimpan di database [nyc_taxi.duckdb](#).

DAG: NYC Taxi Monitor

Tahap ketiga berfungsi untuk monitoring dan deteksi anomali. DAG ini akan memeriksa apakah data hasil transformasi sudah berhasil dan sesuai dengan apa yang diharapkan. Adapun tahapan task nya adalah sebagai berikut :

- [check data freshness](#) akan melakukan pengecekan apakah ada keterlambatan update data
- [validate row counts](#) akan memeriksa jumlah baris antar tabel (misalnya apakah semua record telah terserap)
- [compare with historical](#) akan membandingkan nilai harian dengan data sebelumnya untuk mendeteksi outlier
- [detect anomaly](#) adalah sebuah task yang jika ditemukan penyimpangan signifikan, sistem mengirim email kepada pengguna

Jika semua validasi lolos, sistem akan :

- [generate daily summary](#) yang akan membuat file [summary.csv](#) berisi ringkasan KPI harian.

Dashboard and Analysis

Setelah seluruh pipeline berjalan, hasil data tersimpan dalam bentuk siap untuk dilakukan analisis. Analisis tersebut antara lain :

- [KPI Summary](#) akan membuat metrik kinerja transportasi harian seperti jumlah trip, total pendapatan, dsb.
- [Trend Analysis](#) akan melakukan analisis tren penggunaan taksi berdasarkan waktu
- [Top Zones](#) untuk melihat zona pick up/drop off paling ramai
- [Demand Heatmap](#) digunakan untuk melihat pola permintaan berdasarkan lokasi
- [Payment Breakdown](#) merupakan analisis metode pembayaran pelanggan

Semua analisis akan ditampilkan pada dashboard interaktif, yang dapat diakses oleh pengguna atau analis data.

System Notification

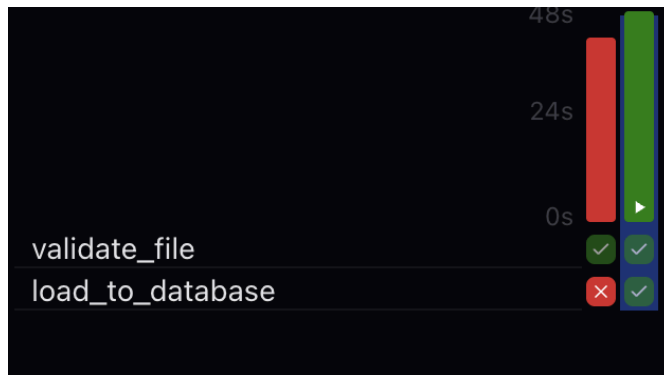
Setiap task di Airflow memiliki dua callback penting diantaranya :

- [onsuccess callback](#) akan mengirimkan email ketika task telah berhasil dieksekusi (misalnya validasi atau load sukses)
- [on failure callback](#) akan mengirimkan email ketika task gagal misalnya file rusak atau gagal load ke Duck DB

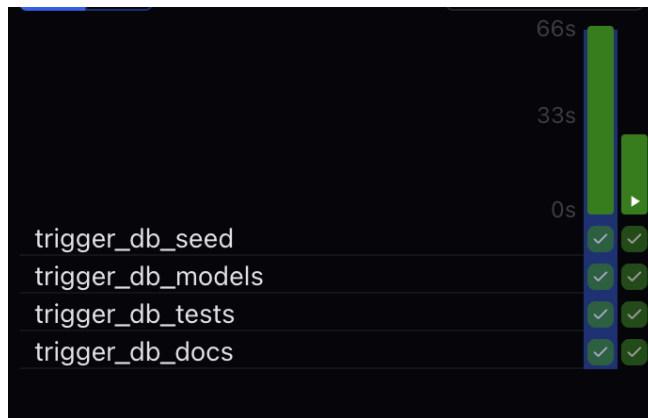
Selain itu, anomaly email akan dikirim secara otomatis jika monitoring mendeteksi data yang menyimpang. Hal ini akan dapat membantu tim jika tim mengetahui lebih cepat ada masalah pada pipeline.

Gantt Chart

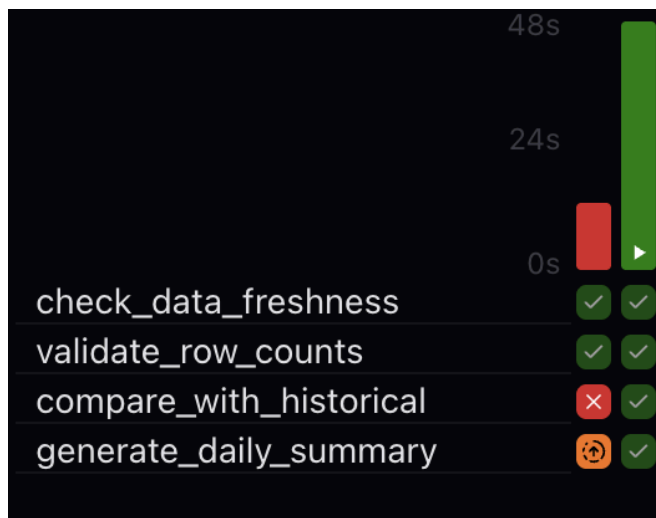
DAG: NYC Taxi Ingestion



DAG: NYC Taxi Transform

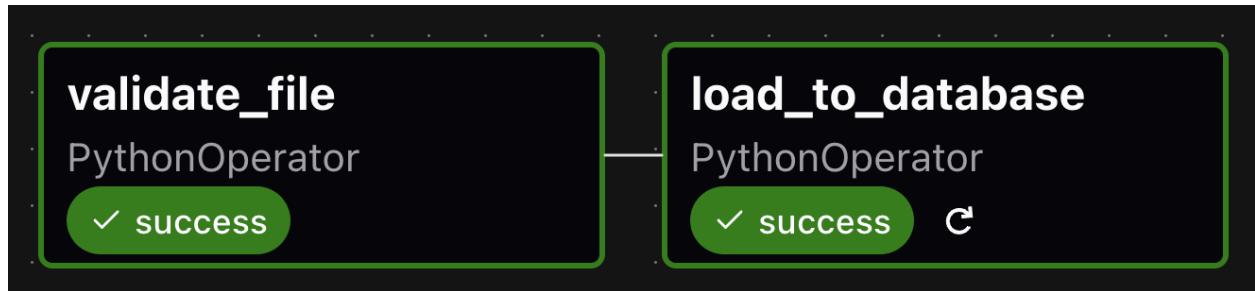


DAG: NYC Taxi Monitor

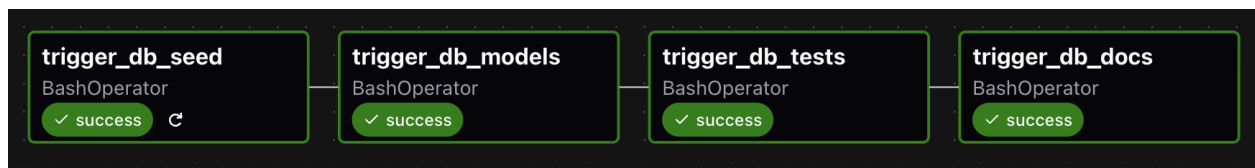


Task Flow

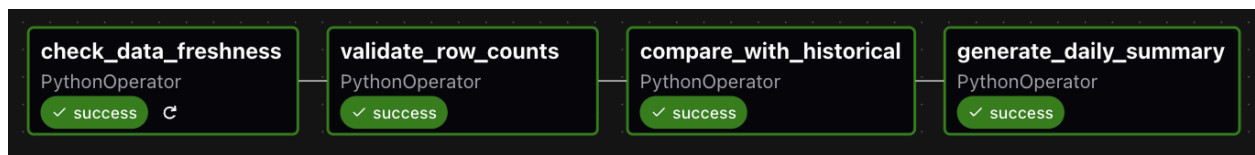
DAG: NYC Taxi Ingestion



DAG: NYC Taxi Transform



DAG: NYC Taxi Monitor



Lineage Graph



Diatas merupakan lineage graph yang menggambarkan pengolahan data yang terjadi pada Duck DB dimana data mengalir dari `nyc_taxiyellow_tripdata` data kemudian menjadi `stg_taxi_trips` lalu menuju `int_trips_enhanced` dan bercabang menjadi tiga bagian diantaranya `fct_trips`, `dim_zones`, `int_daily_metrics`. Pada `int_daily_metrics` kemudian diolah lagi menjadi `agg_daily_stats` dan `daily_anomaly_check`.

F. Summary

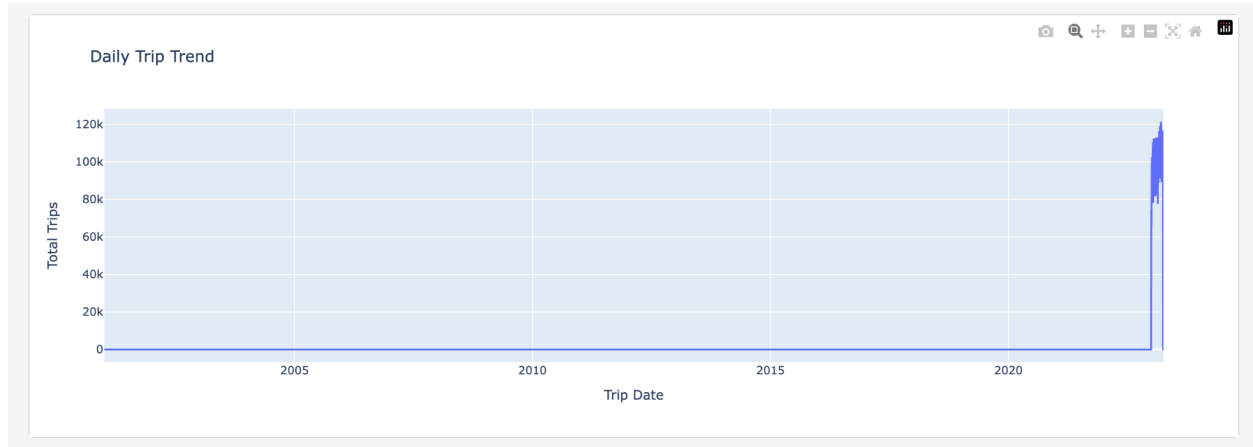
Data Analysis

KPI Summary

Total Trips 9,177,664	Total Revenue \$253,625,402.44	Average Fare \$27.64
--------------------------	-----------------------------------	-------------------------

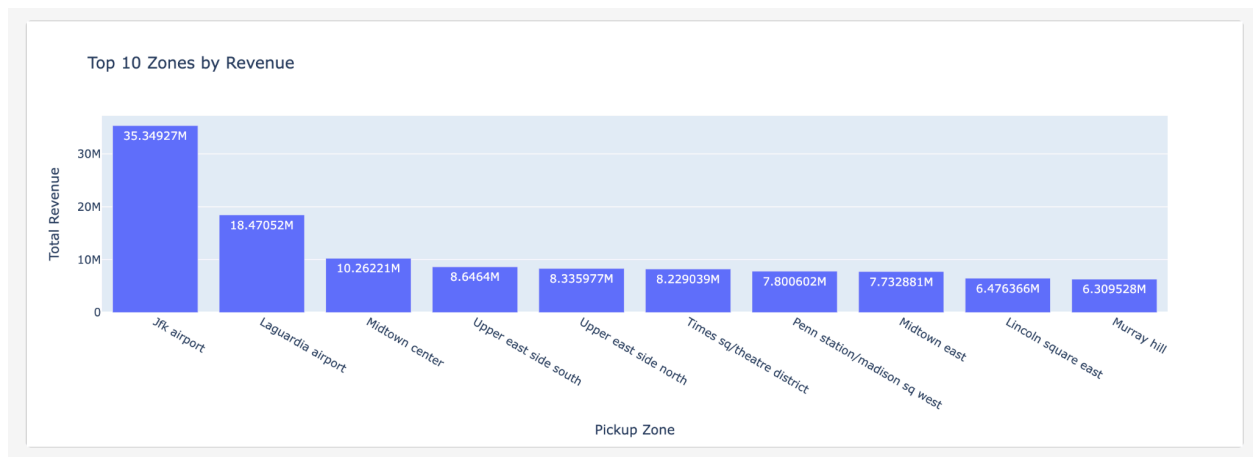
Dari hasil query KPI Summary, terlihat bahwa terdapat 9.177.664 jumlah trips, lalu dilanjutkan dengan jumlah pendapatan yaitu \$253,625,402.44, dan terakhir adalah rata - rata tarif yang berjumlah \$27.64.

Trend Analysis



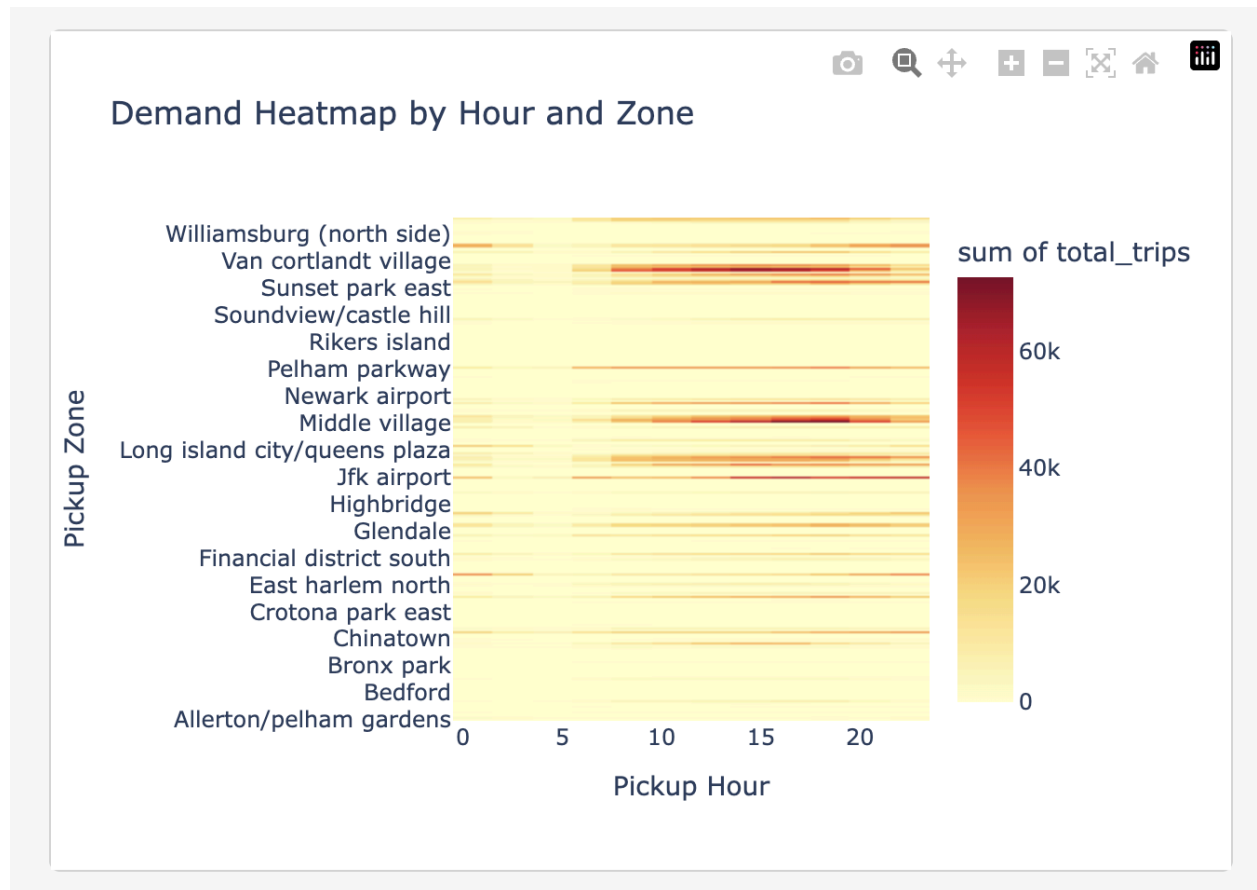
Dapat dilihat pada grafik bahwa terjadi peningkatan pada tahun 2023 awal terhadap total trip.

Top Zones



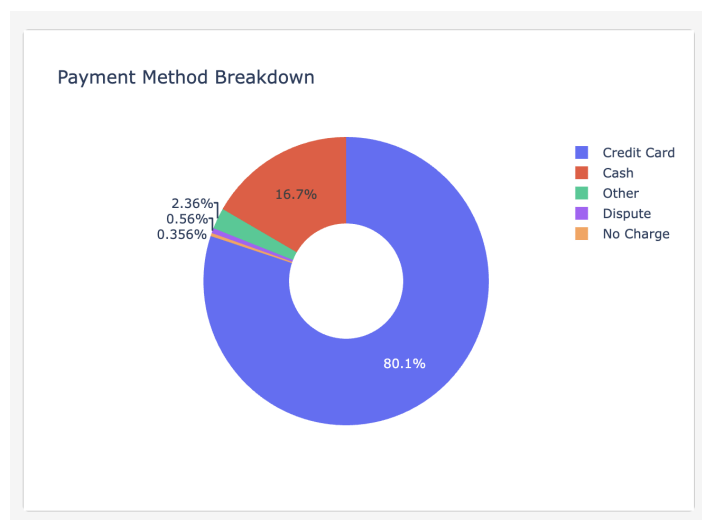
Terdapat total pendapatan tertinggi berada di wilayah JK Airport lalu dilanjutkan di Laguardia Airport, ini membuktikan bahwa pendapatan taksi tertinggi terdapat dengan area yang berdekatan dengan airport.

Demand Heatmap



Dari grafik diatas, dapat disimpulkan bahwa pickup zone paling besar berada di wilayah Van Cortlandt Village pada jam 10.00 pagi hingga jam 20.00 malam.

Payment Breakdown



Dari grafik di atas didapatkan kesimpulan bahwa mayoritas pengguna taksi menggunakan Credit Card sebagai metode pembayaran, lalu dilanjutkan dengan Cash.

Conclusion dan Suggestions

Conclusion

Proyek ini berhasil membangun sebuah ekosistem data pipeline beserta dashboard analitik yang saling terintegrasi untuk membangun sistem analitik menggunakan dataset NYC Taxi Trips. Melalui tahapan staging, transformasi dengan dbt, serta penyimpanan di Duck DB, data mentah yang didapatkan dari [yellow_tripdata](#) berhasil diolah menjadi model analitis seperti [fct_trips](#), [dim_zones](#), dan [agg_daily_stats](#). Hasilnya kemudian divisualisasikan ke dalam bentuk dashboard interaktif menggunakan Python Dash dengan lima komponen utama yaitu KPI Summary, Trend Analysis, Top Zones, Demand Heatmap, dan Payment Breakdown. Analisis ini dapat memberikan wawasan mendalam mengenai pola perjalanan, pendapatan, zona dengan permintaan tertinggi, serta preferensi metode pembayaran penumpang.

Selain itu, penerapan DAG monitoring pada Airflow juga mendukung automasi pemantauan data, yang mencakup validasi data freshness, anomali, serta laporan berkala setiap 6 jam. Dengan desain yang modular dan juga sistem monitoring ini, proses analisis dapat berjalan secara baik, dan dapat kebutuhan decision making untuk sektor transportasi bagi pemilik kepentingan.

Suggestions

Proyek ini sudah memberikan gambaran yang cukup jelas tentang bagaimana data perjalanan dapat diolah dan divisualisasikan agar menghasilkan wawasan yang tepat dan bermanfaat. Namun ada beberapa hal yang bisa dikembangkan lebih lanjut agar sistem ini menjadi lebih lengkap dan lebih mudah digunakan. Salah satunya adalah dengan menambah fitur real-time agar data yang ditampilkan pada dashboard selalu terupdate, sehingga pengguna dapat melihat perubahan pola permintaan atau pendapatan tanpa menunggu proses pembaruan secara manual.

Selain itu, visualisasi pada dashboard juga dapat dibuat lebih interaktif, misalnya dengan menambahkan filter lanjutan dan juga notifikasi otomatis saat ada anomali data terdeteksi. Dari sisi data, penting juga untuk meningkatkan kualitasnya seperti menangani nilai kosong data mendeteksi data yang tidak wajar. Dengan perbaikan - perbaikan tersebut, sistem analitik ini dapat menjadi dasar pengambilan keputusan yang lebih cepat dan akurat bagi pihak-pihak yang berkepentingan.